

Standard Suite

Common classes and commands for most applications.

save options *enum*

"yes" : Save the file.

"no" : Do not save the file.

"ask" : Ask the user whether or not to save the file.

Item Object : A scriptable object.

PROPERTIES

class (type, r/o) : The class of the object.

properties (record) : All of the object's properties.

METHODS

[count](#), [delete](#), [duplicate](#), [exists](#), [get](#), [move](#), [set](#).

Color Object : A color.

Window Object : A window.

ELEMENTS

contained by [application](#).

PROPERTIES

name (text) : The full title of the window.

id (number, r/o) : The unique identifier of the window.

bounds (rectangle) : The bounding rectangle of the window.

closeable (boolean, r/o) : Whether the window has a close box.

titled (boolean, r/o) : Whether the window has a title bar.

index (number) : The index of the window in the back-to-front window ordering.

floating (boolean, r/o) : Whether the window floats.

miniaturizable (boolean, r/o) : Whether the window can be miniaturized.

miniaturized (boolean) : Whether the window is currently miniaturized.

modal (boolean, r/o) : Whether the window is the application's current modal window.

resizable (boolean, r/o) : Whether the window can be resized.

visible (boolean) : Whether the window is currently visible.

zoomable (boolean, r/o) : Whether the window can be zoomed.

zoomed (boolean) : Whether the window is currently zoomed.

selection (list of [Track](#), r/o) : Currently selected tracks

METHODS

[close](#), [print](#), [save](#).

open method : Open an object.

open file : The file(s) to be opened.

print method : Print an object.

print file : The file(s) or document(s) to be printed.

quit *method* : Quit an application.

quit

[**saving**: "yes"/"no"/"ask"] : Specifies whether changes should be saved before quitting.

close *method* : Close an object.

close specifier : the object to close

[**saving**: "yes"/"no"/"ask"] : Specifies whether changes should be saved before closing.

[**savingIn**: file] : The file in which to save the object.

count *method* : Return the number of elements of a particular class within an object.

count specifier : the object whose elements are to be counted

[**each**: type] : The class of objects to be counted.

→ integer : the number of elements

delete *method* : Delete an object.

delete specifier : the object to delete

duplicate *method* : Copy object(s) and put the copies at a new location.

duplicate specifier : the object(s) to duplicate

to: location specifier : The location for the new object(s).

[**withProperties**: record] : Properties to be set in the new duplicated object(s).

exists *method* : Verify if an object exists.

exists specifier : the object in question

→ boolean : true if it exists, false if not

get *method* : Get the data for an object.

get specifier

→ any

make *method* : Make a new object.

make

new: type : The class of the new object.

[**at**: location specifier] : The location at which to insert the object.

[**withData**: any] : The initial data for the object.

[**withProperties**: record] : The initial values for properties of the object.

→ specifier : to the new object

move *method* : Move object(s) to a new location.

move specifier : the object(s) to move

to: location specifier : The new location for the object(s).

save *method* : Save an object.

save specifier : the object to save, usually a document or window

[**in:** file] : The file in which to save the object.

[**as:** text] : The file type in which to save the data.

set *method* : Set an object's data.

set specifier

to: any : The new value.

Swinsian Suite

Picture *Object* : picture data

Application *Object* : The application

ELEMENTS

contains [windows](#), [playlists](#), [smartPlaylists](#), [normalPlaylists](#), [tracks](#), [audioDevices](#).

PROPERTIES

name (text, r/o) : The name of the application.

frontmost (boolean, r/o) : Is this the frontmost (active) application?

version (text, r/o) : The version of the application.

playerPosition (integer) : the player's position within the currently playing track in seconds.

currentTrack ([Track](#), r/o) : the currently playing track

soundVolume (number) : the volume. (0 minimum, 100 maximum)

playerState ("stopped"/"playing"/"paused", r/o) : are we stopped, paused or still playing?

playbackQueue ([Queue](#), r/o) : the currently queued tracks

outputDevice ([AudioDevice](#)) : current audio output device

repeatQueue (boolean) : repeat playback

repeatSingle (boolean) : repeat single track

stopAfterEveryTrack (boolean) : stop playback at the end of every track

stopAfterTrack (boolean) : stop after the current track

musicLibrary ([Library](#), r/o) : the music library

shuffle ("track shuffle"/"album shuffle"/"none") : shuffle mode

METHODS

[quit](#), [play](#), [pause](#), [stop](#), [playpause](#), [nextTrack](#), [previousTrack](#), [notify](#), [rescanTags](#), [findTrack](#).

LibraryTrack *Object* [inh. [Track](#) > [Item](#)]

Playlist *Object* [inh. [Item](#)] : generic playlist type, subclasses include smart playlist and normal playlist

ELEMENTS

contains [tracks](#); contained by [application](#), [playlistFolders](#).

PROPERTIES

name (text) : the name of the playlist

smart (boolean, r/o) : is this a smart playlist

Library *Object*, *pl libraries* [inh. [Item](#)]

ELEMENTS

contains [tracks](#).

iPodTrack *Object* [inh. [Track](#) > [Item](#)]

PROPERTIES

ipodName (text, r/o) : the name of the iPod this track is on

Track *Object* [inh. [Item](#)] : a music track

ELEMENTS

contained by [application](#), [playlists](#), [libraries](#), [queues](#), [normalPlaylists](#).

PROPERTIES

album (text) : the album of the track

artist (text) : the artist

composer (text) : the composer

genre (text) : the genre

time (text, r/o) : the length of the track in text format as MM:SS

year (integer) : the year the track was recorded

dateAdded (date, r/o) : the date the track was added to the library

duration (real, r/o) : the length of the track in seconds

location (text, r/o) : location on disk

ipodTrack (boolean, r/o) : TRUE if the track is on an iPod

name (text) : the title of the track (same as title)

bitRate (integer, r/o) : the bitrate of the track

kind (text, r/o) : a text description of the type of file the track is

rating (number) : Track rating. 0-5

trackNumber (integer) : the Track number

fileSize (integer, r/o) : file size in bytes

albumArt ([Picture](#), r/o) : the album artwork

artFormat (text, r/o) : the data format for this piece of artwork. text that will be "PNG" or "JPEG". getting the album art property first will mean this information has

been retrieved already, otherwise the tags for the file will have to be re-read

discNumber (number) : the disc number
discCount (number) : the total number of discs in the album
id (text, r/o) : uuid
albumArtist (text) : the album artist
albumArtistOrArtist (text, r/o) : the album artist of the track, or is none is set, the artist
compilation (boolean) : compilation flag
title (text) : track title (the same as name)
comment (text) : the comment
dateCreated (date, r/o) : the date created
channels (integer, r/o) : audio channel count
sampleRate (integer, r/o) : audio sample rate
bitDepth (integer, r/o) : the audio bit depth
lastPlayed (date) : date track was last played
lyrics (text) : track lyrics
path (text, r/o) : POSIX style path
grouping (text) : grouping
publisher (text) : the publisher
conductor (text) : the conductor
description (text) : the description
encoder (text, r/o) : the encoder
copyright (text, r/o) : the copyright
catalogNumber (text) : the catalog number
dateModified (date, r/o) : the date modified
playCount (integer) : the play count
trackCount (number) : the total number of tracks in the album
enabled (boolean) : the enabled state of the track
sortArtist (text) : the sort artist
sortAlbum (text) : the sort album
sortAlbumArtist (text) : the sort album artist
bpm (number) : the bpm
barcode (text) : the barcode

METHODS

[add](#).

Queue *Object* [inh. [Item](#)] : The playback queue

ELEMENTS

contains [tracks](#).

SmartPlaylist *Object* [inh. [Playlist](#) > [Item](#)] : a smart playlist

NormalPlaylist *Object* [inh. [Playlist](#) > [Item](#)] : a normal, non-smart, playlist

PROPERTIES

id (text, r/o) : uuid

METHODS

[add](#).

PlaylistFolder *Object* [inh. [Playlist](#) > [Item](#)] : folder of playlists

ELEMENTS

contains [playlists](#).

PROPERTIES

id (text, r/o) : uuid

AudioDevice *Object* : an audio output device

ELEMENTS

contained by [application](#).

PROPERTIES

name (text, r/o) : device name

id (text) : uuid

play *method* : begin playing the current playlist

play

pause *method* : pause playback

pause

nextTrack *method* : skip to the next track in the current playlist

nextTrack

stop *method* : stop playback

stop

search *method* : search a playlist for tracks matching a string

search

playlist: [Playlist](#) : the playlist to search

for: text : the search string

→ list of [Track](#) : the tracks found

previousTrack *method* : skip back to the previous track

previousTrack

playpause *method* : toggle play/pause

playpause

add *method* : add a track to a playlist

add

tracks: list of [Track](#)

to: [NormalPlaylist](#) : the playlist

notify *method* : show currently playing track notification

notify

rescanTags *method* : rescan tags on tracks

rescanTags list of [Track](#) : tracks to rescan

findTrack *method* : Finds tracks for the given path

findTrack text : POSIX style path

→ list of [Track](#) : matching tracks

remove *method* : remove tracks from a playlist

remove

tracks: list of [Track](#) : tracks to remove

from: [NormalPlaylist](#)